บทที่ 8

การเก็บข้อมูลในเครื่องและการเชื่อมต่อเครือข่าย API

ในบทเรียนนี้ เราจะเจาะลึกระบบการจัดการข้อมูลที่มีความมั่นคงถาวรขึ้นไปอีกขั้น โดยเรียนรู้การเขียนบันทึกข้อมูลและเซสชันไว้ในพื้นที่หน่วยความจำของมือถือด้วยคอมโพเนนต์ AsyncStorage รวมถึงสถาปัตยกรรมการติดต่อฐานข้อมูลกับเครื่องแม่ข่ายเซิร์ฟเวอร์ภายนอกผ่านเครือข่ายอินเทอร์เน็ตด้วยคำสั่ง Fetch API ทั้งรูปแบบการดึงข้อมูลและการส่งโพสต์ข้อมูลใหม่ พร้อมแนวทางรับมือกรณีสัญญาณอินเทอร์เน็ตล่ม

8.1 ความเข้าใจและการติดตั้ง AsyncStorage

แอปพลิเคชันโดยทั่วไป หากเก็บตัวแปรข้อมูลไว้บนระดับสถานะ (State) ของ React ข้อมูลเหล่านั้นจะสูญหายทันทีที่ผู้ใช้ปิดแอป หรือเครื่องโทรศัพท์แบตเตอรี่หมด เพื่อรักษาสภาพความคงอยู่ของข้อมูล (Data Persistence) เช่น การจดจำชื่อโปรไฟล์ หรือโทเค็นการล็อกอิน (Auth Token) เราจะใช้หน่วยเก็บข้อมูลถาวรประเภทคีย์-ค่า (Key-Value Storage) ที่เรียกว่า **AsyncStorage** (React Native Async Storage, 2024)

คุณสมบัติหลักของ AsyncStorage:

• **Asynchronous (ทำงานแบบไม่ประสานเวลา):** ทุกคำสั่งการอ่าน เขียน หรือลบข้อมูลจะทำงานเบื้องหลังโดยไม่บล็อกการทริกเกอร์แอนิเมชันหรือการวาดหน้าจอ (คืนค่าเป็น Promise)

• **Persistent (คงอยู่ตลอดไป):** ข้อมูลจะถูกจัดเก็บถาวรในตัวเครื่องจนกว่าแอปพลิเคชันจะถูกถอนการติดตั้ง (Uninstall) หรือมีการรันคำสั่งสั่งล้างข้อมูล

• **String-Only (จัดเก็บเฉพาะสตริง):** ตัวจัดเก็บนี้บันทึกได้เฉพาะข้อมูลที่เป็นข้อความตัวหนังสือ (String) เท่านั้น หากมีข้อมูลประเภทอื่น เช่น ตัวเลข หรือออบเจกต์ จำเป็นต้องมีกระบวนการแปลงค่าก่อนเสมอ

คำสั่งติดตั้งสำหรับ Expo SDK 54:

```
npx expo install @react-native-async-storage/async-storage
```

8.2 การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations)

ในการเรียกเขียนหรืออ่านข้อมูล เราจำเป็นต้องเรียกใช้คำสั่งผ่านบล็อกคำสั่ง async/await เสมอ เพื่อรับประกันว่าข้อมูลถูกเขียนหรืออ่านเสร็จก่อนจะนำผลลัพธ์ไปประมวลผลต่อ

1. การเขียนหรืออัปเดตข้อมูล (setItem)

```
import AsyncStorage from '@react-native-async-storage/async-storage';

const saveData = async (value: string) => {
  try {
    await AsyncStorage.setItem('user_name', value);
    console.log('บันทึกข้อมูลเรียบร้อยแล้ว');
  } catch (error) {
    console.error('บันทึกผิดพลาด:', error);
  }
};
```

2. การอ่านข้อมูลเดิมกลับมาใช้ (getItem)

```
const loadData = async () => {
  try {
    const value = await AsyncStorage.getItem('user_name');
    if (value !== null) {
      console.log('ข้อมูลที่อ่านได้:', value);
    }
  } catch (error) {
    console.error('อ่านข้อมูลผิดพลาด:', error);
  }
};
```

3. การลบข้อมูลบางชิ้น (removeItem)

```
const removeData = async () => {
  try {
    await AsyncStorage.removeItem('user_name');
    console.log('ลบข้อมูลสำเร็จ');
  } catch (error) {
    console.error('ลบข้อมูลผิดพลาด:', error);
  }
};
```

8.3 การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization)

เนื่องจาก AsyncStorage สนับสนุนการบันทึกเฉพาะชนิดข้อมูล String หากเราส่งโครงสร้างประเภทออบเจกต์ (Object) หรืออาร์เรย์ (Array) ไปตรง ๆ ระบบจะไม่สามารถทำงานได้ถูกต้อง เราจึงต้องแปลงรูปแบบข้อมูลให้เป็นรูปแบบสตริงด้วยการทำ Serialization

- **ก่อนทำการบันทึก:** แปลงออบเจกต์ให้เป็นสตริงด้วย `JSON.stringify(object)`
- **หลังจากดึงข้อมูลกลับมา:** แปลงสตริงกลับไปเป็นออบเจกต์ด้วย `JSON.parse(string)`

```
import AsyncStorage from '@react-native-async-storage/async-storage';

interface UserProfile {
  name: string;
  age: number;
  skills: string[];
}

// 1. ฟังก์ชันบันทึกข้อมูลประเภทออบเจกต์ลงเครื่อง
export const saveUserProfile = async (profile: UserProfile) => {
  try {
    const jsonString = JSON.stringify(profile); // แปลง Object เป็น JSON String
    await AsyncStorage.setItem('user_profile', jsonString);
  } catch (e) {
    console.error('บันทึกออบเจกต์ผิดพลาด:', e);
  }
};

// 2. ฟังก์ชันดึงและแปลงข้อมูลกลับมาเป็นออบเจกต์เดิม
export const getUserProfile = async (): Promise<UserProfile | null> => {
  try {
    const jsonString = await AsyncStorage.getItem('user_profile');
    return jsonString != null ? JSON.parse(jsonString) : null; // แปลง JSON String กลับเป็น Object
  } catch (e) {
    console.error('ดึงออบเจกต์ผิดพลาด:', e);
    return null;
  }
};
```

8.4 การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET)

นอกเหนือจากการเก็บข้อมูลภายในเครื่องแล้ว โมบายแอปพลิเคชันส่วนใหญ่จำเป็นต้องมีการทำงานร่วมกับระบบเซิร์ฟเวอร์หลังบ้านผ่านสถาปัตยกรรม REST API คำสั่งมาตรฐานที่รองรับการใช้งานใน Expo โดยไม่ต้องติดตั้งไลบรารีภายนอกเพิ่มคือ **`fetch()`** (MDN Web Docs, 2024)

ตัวอย่างคำสั่งดึงข้อมูลล่าสุด (GET) จากเซิร์ฟเวอร์ JSONPlaceholder:

```
import { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator } from 'react-native';

export default function ApiGetExample() {
  const [data, setData] = useState<any>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
  // ฟังก์ชันดึงข้อมูลแบบ Asynchronous
    const fetchPost = async () => {
      try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
        const json = await response.json(); // แปลงการตอบสนองกลับเป็นออบเจกต์
        setData(json);
      } catch (error) {
        console.error('เกิดข้อผิดพลาดในการดึงข้อมูล:', error);
      } finally {
        setLoading(false);
      }
    };

    fetchPost();
  }, []);

  if (loading) return <ActivityIndicator size="large" />;

  return (
    <View style={{ padding: 20 }}>
      {data && (
        <>
          <Text style={{ fontWeight: 'bold', fontSize: 18
}}>{data.title}</Text>
          <Text style={{ marginTop: 10 }}>{data.body}</Text>
        </>
   )}
    </View>
 );
}
```

8.5 การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST)

การสร้างข้อมูลชิ้นใหม่บนเซิร์ฟเวอร์ (เช่น การส่งแบบฟอร์มสมัครสมาชิก, การตั้งกระทู้ใหม่) จะทำงานผ่านการส่ง Request ในรูปแบบ **HTTP POST** ซึ่งเราต้องระบุอ็อพชันหลัก 3 อย่างในพารามิเตอร์ที่สองของคำสั่ง `fetch()`:

1. `method`: กำหนดให้เป็นสตริง `'POST'`
2. `headers`: กำหนดประเภทข้อมูลที่ส่งไป เช่น `'Content-Type': 'application/json'`
3. `body`: ข้อมูล JSON ที่ถูกจัดทำการแปลงเป็นสตริงด้วย `JSON.stringify(data)`

```
import { useState } from 'react';
import { View, TextInput, Button, Alert, ActivityIndicator, StyleSheet }
from 'react-native';

export default function SendPostScreen() {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
 const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async () => {
    if (!title || !body) {
      Alert.alert('กรุณากรอกข้อมูล', 'จำเป็นต้องกรอกทั้งหัวเรื่องและเนื้อหา');
      return;
    }

    try {
      setSubmitting(true);

   // ส่งข้อมูล POST Request ไปยังเป้าหมายภายนอก
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json; charset=UTF-8',
    },
        body: JSON.stringify({
          title: title,
          body: body,
          userId: 1,
        }),
      });

      if (response.ok) {
        const result = await response.json();
    // แสดงการตอบรับความสำเร็จจากเซิร์ฟเวอร์ (เช่น รหัสไอดี 101)
        Alert.alert('ส่งข้อมูลสำเร็จ!', `สร้างโพสต์ใหม่รหัส ID: ${result.id}`);
        setTitle('');
        setBody('');
```

```
      } else {
        throw new Error('เกิดข้อผิดพลาดจากฝั่งเซิร์ฟเวอร์');
      }
    } catch (error: any) {
      Alert.alert('การส่งข้อมูลล้มเหลว', error.message || 'กรุณาตรวจสอบสัญญาณ
อินเทอร์เน็ต');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="กรอกหัวเรื่อง (Title)"
        value={title}
        onChangeText={setTitle}
   />
      <TextInput
        style={[styles.input, styles.textArea]}
        placeholder="กรอกเนื้อหาโพสต์ (Body)"
        value={body}
        onChangeText={setBody}
        multiline
   />
      {submitting ? (
        <ActivityIndicator size="small" color="#38bdf8" />
   ) : (
        <Button title="ส่งโพสต์ขึ้นเซิร์ฟเวอร์" onPress={handleSubmit} />
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  container: { padding: 20 },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    borderRadius: 8,
    marginBottom: 12,
  },
  textArea: {
    height: 100,
    textAlignVertical: 'top',
  },
});
```

8.6 ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด (Error Handling)

สุนทรียภาพที่ดีของโมบายแอปพลิเคชัน คือการแจ้งให้ผู้ใช้งานรับรู้อย่างชัดเจนว่าแอปพลิเคชันกำลังดำเนินกิจกรรมอะไรอยู่ และสามารถดักจับสถานการณ์เมื่อไม่สามารถติดต่อเซิร์ฟเวอร์ได้

แนวทางการจัดการสถานะข้อมูล:

1. **สถานะ Loading (`isLoading`):** ตั้งค่าเป็น `true` ก่อนที่กระบวนการ `fetch()` จะเริ่มขึ้น และตั้งคืนเป็น `false` เสมอในบล็อก `finally` เพื่อปิดไอคอนตัวหมุน
2. **สถานะ Error (`error`):** ใช้บล็อก `try-catch` ในการสกัดกั้นข้อผิดพลาดของอินเทอร์เน็ต หากพบอาการขัดข้องให้เก็บออบเจกต์ข้อผิดพลาดไว้แสดงหน้าจอแจ้งเตือนปุ่มรีเฟรช (Retry Button)
3. **ตรวจสอบรหัสสถานะความสำเร็จ (`response.ok`):** ในบางครั้งเครือข่ายสัญญาณดีมากแต่ฐานข้อมูลปลายทางมีปัญหา (เช่น ส่งรหัส 404 หรือ 500 กลับมา) ตัวคำสั่ง `fetch` จะไม่ทริกเกอร์เข้าบล็อก `catch` อัตโนมัติ เราจึงต้องเขียนดักเช็คค่าของ `response.ok` (มีค่าเป็น true เมื่อ status อยู่ในช่วง 200-299) ด้วยตัวเอง

8.7 การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน (Auth Token Caching)

หนึ่งในการประยุกต์ใช้งาน AsyncStorage ที่พบเจอได้บ่อยที่สุดในโปรเจกต์เชิงพาณิชย์คือการจัดการระบบล็อกอินเข้าใช้งานของสมาชิก โดยหลังจากผู้ใช้ทำการล็อกอินสำเร็จ โทเค็น (Token) จะถูกเก็บไว้ใน AsyncStorage เพื่อที่การเปิดแอปใช้งานครั้งถัดไป ผู้ใช้จะไม่ต้องกรอกรหัสผ่านซ้ำอีกรอบ

แผนภาพการไหลของระบบจดจำล็อกอิน:

```
graph TD
    A[เปิดแอปพลิเคชัน] --> B{มี token ใน AsyncStorage หรือไม่?}
    B -- มี token --> C[ข้ามไปที่หน้า Dashboard / หน้าหลัก]
    B -- ไม่มี token --> D[แสดงหน้า Login ให้กรอกข้อมูล]
    D --> E[กรอกรหัสผ่านส่ง API สำเร็จ]
    E --> F[เซฟ token ลง AsyncStorage]
    F --> C
```

ตัวอย่างโครงสร้างโค้ดการจัดการตรวจสอบสิทธิ์ตอนเริ่มต้นเปิดแอป:

```
import { useEffect, useState } from 'react';
```

```
import { View, Text, Button, StyleSheet, ActivityIndicator } from 'react-
native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function AppSessionManager() {
  const [appReady, setAppReady] = useState(false);
  const [userToken, setUserToken] = useState<string | null>(null);

 // ตรวจเช็คโทเค็นที่ถูกแคชไว้ตอนเปิดแอป
  useEffect(() => {
    const checkToken = async () => {
      try {
        const token = await AsyncStorage.getItem('user_auth_token');
        if (token) {
          setUserToken(token); // อัปเดตสเตตหากพบข้อมูลเก่า
        }
      } catch (e) {
        console.error(e);
      } finally {
        setAppReady(true);
      }
    };
    checkToken();
  }, []);

 // ฟังก์ชันจำลองล็อกอิน
  const handleLogin = async () => {
    const mockToken = 'jwt-secure-token-123456';
    await AsyncStorage.setItem('user_auth_token', mockToken);
    setUserToken(mockToken);
  };

 // ฟังก์ชันล็อกเอาต์
  const handleLogout = async () => {
    await AsyncStorage.removeItem('user_auth_token');
    setUserToken(null);
  };

  if (!appReady) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" />
      </View>
  );
  }

  return (
    <View style={styles.container}>
      {userToken ? (
        <View>
          <Text style={styles.text}>🔓 ยินดีต้อนรับกลับ! เข้าสู่ระบบสำเร็จ</Text>
```

```
            <Text style={styles.subText}>Token: {userToken}</Text>
            <Button title="ออกจากระบบ (Logout)" color="#ef4444"
onPress={handleLogout} />
          </View>
    ) : (
          <View>
            <Text style={styles.text}>🔒 กรุณาลงชื่อเข้าใช้งาน</Text>
            <Button title="ลงชื่อเข้าใช้จำลอง (Login)" onPress={handleLogin} />
          </View>
    )}
      </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center',
backgroundColor: '#f1f5f9' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  text: { fontSize: 18, fontWeight: 'bold', marginBottom: 10, textAlign:
'center' },
  subText: { fontSize: 13, color: '#64748b', marginBottom: 20, textAlign:
'center' },
});
```

สรุปท้ายบทเรียน

ความเข้าใจและการติดตั้ง AsyncStorage: AsyncStorage เป็นเครื่องมือสำคัญในการรักษาความคงอยู่ของข้อมูล in ระดับอุปกรณ์ประเภทคีย์-ค่า (Key-Value) แบบทำงานเบื้องหลังโดยไม่รบกวนหน้าจอ (Asynchronous) ซึ่งข้อมูลจะอยู่ถาวรยกเว้นแต่จะถอนติดตั้งหรือสั่งล้างข้อมูล โดยใน Expo SDK 54 เราสามารถติดตั้งมาใช้งานได้สะดวกผ่านคำสั่ง npx expo install เพื่อนำมาจัดเก็บข้อมูลถาวร เช่น การตั้งค่าหรือเซสชันล็อกอิน

การทำงานพื้นฐานกับ AsyncStorage (CRUD Operations): กระบวนการทำงานพื้นฐานในการเก็บและดึงข้อมูล AsyncStorage จะใช้คำสั่ง async/await ในการประสานงาน โดยมีคำสั่งหลักที่สำคัญคือ setItem สำหรับเขียนหรืออัปเดตข้อมูล, getItem เพื่อดักเรียกอ่านข้อมูลเดิมกลับมาประมวลผลต่อ และ removeItem สำหรับการสั่งลบข้อมูลแต่ละรายการที่ระบุค่าคีย์ไว้

การบันทึกและดึงข้อมูลออบเจกต์ซับซ้อน (JSON Serialization): เนื่องจาก AsyncStorage รองรับข้อมูลชนิดข้อความสตริงเท่านั้น เมื่อต้องการจัดเก็บข้อมูลซับซ้อนประเภทออบเจกต์หรืออาร์เรย์ ผู้พัฒนาจำเป็นต้องแปลงข้อมูลเป็นสตริงผ่านกระบวนการ

JSON.stringify(object) ก่อนนำลงไปเก็บบันทึก และทำการแปลงข้อมูลสตริงสกัดกลับเป็นออบเจกต์ดั้งเดิมผ่าน JSON.parse(string) หลังเรียกขึ้นมาใช้งาน

การดึงข้อมูลจาก API ด้วย Fetch (HTTP GET): การประมวลผลดึงข้อมูล API ภายนอกมายังแอปพลิเคชันจะทำงานผ่านฟังก์ชัน fetch() ของระบบเพื่อยิง GET Request ไปยังจุดปลายของเซิร์ฟเวอร์ปลายทาง โดยเรียกใช้คำสั่งควบคู่กับ useEffect ในการดึงข้อมูลสดเมื่อเริ่มรันแอป และทำการแปลงค่าการตอบสนองเป็นโครงสร้างออบเจกต์ด้วยเมธอด .json() ก่อนนำไปแจกจ่าย

การส่งข้อมูลขึ้นเซิร์ฟเวอร์ด้วย Fetch (HTTP POST): การส่งข้อมูลใหม่ขึ้นฝั่งเซิร์ฟเวอร์ภายนอกใช้วิธีส่งแบบ HTTP POST ผ่านการระบุอ็อพชันเพิ่มเติมใน fetch() โดยระบุ method: 'POST', ระบุประเภท content-type เป็น JSON ในส่วนของ headers และส่งข้อมูลหลักที่ถูกแปลงเป็นสตริงด้วย JSON.stringify ในส่วนของ body ซึ่งช่วยให้เราสามารถสมัครสมาชิกหรือโพสต์ข้อมูลได้สำเร็จ

ระบบจัดการสถานะ Loading และการดักจับข้อผิดพลาด: การเพิ่มสัมผัสความลื่นไหลในการใช้บริการเน็ตเวิร์กจำต้องมีการจัดการสถานะ Loading เพื่อควบคุมการแสดง ActivityIndicator และครอบคลุมระบบดักจับปัญหา Error Handling ผ่านบล็อก try-catch เพื่อดักจับสัญญาณอินเทอร์เน็ตล่ม โดยต้องทำการตรวจสอบค่าผลลัพธ์ผ่าน response.ok ควบคู่กันเพื่อป้องกันกรณีเซิร์ฟเวอร์ส่งข้อผิดพลาดกลับมา

การประยุกต์ใช้ในการทำระบบจำเซสชันการล็อกอิน: การประยุกต์ใช้ AsyncStorage และเน็ตเวิร์ก API ที่มีความสำคัญในแอปพลิเคชันเชิงพาณิชย์คือระบบเซสชันการเข้าสู่ระบบ โดยหลังจากยืนยันตัวตนสำเร็จแล้วจะนำ Auth Token ไปบันทึกลงในเครื่อง และเมื่อเปิดแอปใหม่จะสามารถดักจับโทเค็นดังกล่าวมาล็อกอินอัตโนมัติ ช่วยลดความซับซ้อนให้ผู้ใช้งานไม่ต้องป้อนข้อมูลซ้ำซ้อน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้ความเข้าใจเกี่ยวกับการใช้งาน AsyncStorage ในการบันทึกข้อมูลถาวรประเภทข้อความและออบเจกต์ซับซ้อน, การดึงและส่งข้อมูลด้วย Fetch API (HTTP GET & POST), การตรวจจับสถานะการตอบสนองความสำเร็จผ่าน `response.ok`, และการทำระบบจดจำล็อกอินด้วยโทเค็น

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายคุณลักษณะการจัดเก็บข้อมูลของไลบรารี AsyncStorage ใน React Native ได้ถูกต้องที่สุด?

ก. บันทึกข้อมูลประเภทอ็อบเจกต์ของภาษารันไทม์ไว้ในคลาวด์เซิร์ฟเวอร์แบบเรียลไทม์

ข. จัดเก็บข้อมูลแบบถาวรในรูปแบบกุญแจ-ค่า (Key-Value) เฉพาะข้อมูลชนิดข้อความ (String) บนพื้นที่ตัวเครื่องของผู้ใช้

ค. เก็บข้อมูลไว้ในหน่วยความจำ RAM ซึ่งจะหายไปทันทีเมื่อปิดหน้าต่างแอปพลิเคชัน

ง. บันทึกรูปภาพและวิดีโอต้นฉบับลงในแกลเลอรีภาพหลักของสมาร์ทโฟน

ข้อ 2: เมื่อผู้ใช้ปิดแอปพลิเคชันหรือทำการรีสตาร์ทตัวเครื่องโทรศัพท์ใหม่ ข้อมูลที่บันทึกไว้ใน AsyncStorage จะได้รับผลกระทบอย่างไร?

ก. ข้อมูลจะถูกเข้ารหัสสุ่มและย้ายไปฝากไว้ที่เซิร์ฟเวอร์ของ Expo

ข. ข้อมูลจะคงอยู่เหมือนเดิมอย่างปลอดภัย เนื่องจากข้อมูลถูกเก็บถาวรลงในไดรฟ์ของระบบปฏิบัติการเครื่อง

ค. ข้อมูลจะสูญหายทันทีเนื่องจาก AsyncStorage ไม่รองรับหน่วยความจำระยะยาว

ง. ข้อมูลจะกลายเป็นสัญลักษณ์เครื่องหมายคำถามทั้งหมดจนกว่าจะเปิดสัญญาณ WiFi

ข้อ 3: หากต้องการเขียนทับบันทึกหรือส่งชุดข้อมูลสตริงใหม่ไปเก็บใน AsyncStorage ผู้พัฒนาแอปพลิเคชันต้องเรียกใช้งานเมธอดใด?

ก. `AsyncStorage.getItem`

ข. `AsyncStorage.removeItem`

ค. `AsyncStorage.setItem`

ง. `AsyncStorage.clear`

ข้อ 4: ข้อใดคือพฤติกรรมที่เกิดขึ้นหากคุณส่งออบเจกต์ผู้ใช้งาน เช่น `const user = { name: "สมชาย", age: 30 };` เข้าไปที่คำสั่ง `AsyncStorage.setItem('user_data', user)` โดยตรงโดยไม่มีการแปลงรูปแบบข้อมูล?

ก. ระบบจะบันทึกข้อมูลออบเจกต์ได้สมบูรณ์และใช้งานได้ปกติ

ข. ระบบจะเกิดข้อผิดพลาดในการทำงาน หรือข้อมูลจะถูกแปลงกลายสตริงข้อความแทนที่ตัวตนจริงเป็นค่า `"[object Object]"` ซึ่งไม่สามารถนำกลับมาแกะข้อมูลด้านในได้

ค. ระบบจะอัปโหลดข้อมูลตัวแปรนี้ขึ้นสู่เซิร์ฟเวอร์ของเฟซบุ๊กโดยอัตโนมัติ

ง. ระบบจะเปลี่ยนสไตล์หน้าจอปุ่มสัมผัสของเครื่องเป็นสีแดงเพื่อเตือนภัย

ข้อ 5: วิธีการและเครื่องมือคู่ใดที่เหมาะสมที่สุดในการบันทึกข้อมูลออบเจกต์ข้ามหน้าด้วย AsyncStorage เพื่อไม่ให้สูญเสียคุณสมบัติข้อมูลลูกด้านใน?

ก. ใช้ `JSON.stringify(object)` แปลงสตริงก่อนจัดเก็บ และใช้ `JSON.parse(string)` แกะกลับเป็นออบเจกต์ตอนดึงข้อมูลกลับมา

ข. ใช้คำสั่ง `toString()` ในการบันทึก และใช้ `Number()` ในการเรียกรับค่ากลับคืนมา

ค. ใช้ระบบ `AsyncStorage.clear()` ทั้งก่อนเขียนและหลังเขียนข้อมูลใหม่

ง. ไม่จำเป็นต้องแปลงค่าใด ๆ เพียงใส่คำสั่ง `await` คุมไว้ระบบเนทีฟจะสืบประเภทให้เอง

ข้อ 6: เมื่อเรียกคำสั่ง `fetch()` เพื่อเชื่อมข้อมูลผ่านอินเทอร์เน็ต หากเซิร์ฟเวอร์หลังบ้านส่งรหัสสถานะข้อผิดพลาด เช่น 404 `Not Found` หรือ 500 `Internal Server Error` กลับมา ตัวกลไก Promise ของ Fetch จะมีพฤติกรรมอย่างไร?

ก. จะสั่ง Reject เข้าสู่บล็อกข้อผิดพลาด `catch` โดยอัตโนมัติ

ข. จะยังทำงานส่งต่อความสำเร็จ (Resolve) ตามปกติโดยไม่วิ่งไปที่บล็อก `catch` ซึ่งผู้พัฒนาต้องตรวจสอบความสมบูรณ์ผ่านพร็อพส์ `response.ok` ด้วยตนเอง

ค. จะปิดระบบเครือข่ายของแอปเพื่อป้องกันความเสียหาย

ง. จะทำลายการเชื่อมต่อสเตตสถานะล็อกอินของผู้ใช้ทั้งหมดทิ้งทันที

ข้อ 7: พร็อพส์ (Properties) ตัวใดของ Response Object ที่ตอบกลับจากคำสั่ง `fetch()` ซึ่งระบุค่าสถานะความสำเร็จของเครือข่ายที่มีรหัส HTTP Status อยู่ในช่วง 200 ถึง 299?

ก. `response.statusText`

ข. `response.headers`

ค. `response.ok`

ง. `response.bodyUsed`

ข้อ 8: การเขียนคำสั่งส่งข้อมูลแบบ HTTP POST ด้วยคำสั่ง Fetch API ข้อใดระบุพารามิเตอร์การจัดส่งข้อมูลในช่อง `fetch(url, options)` ได้ถูกต้องและครบถ้วนตามหลักการพัฒนา?

ก. `options: { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(data) }`

ข. `options: { method: 'GET', headers: { 'Accept': 'text/plain' }, body: data }`

ค. `options: { method: 'DELETE', body: data.toString() }`

ง. `options: { method: 'POST', body: data }`

ข้อ 9: ในขั้นตอนจัดการการตอบสนองผู้ใช้ขณะที่ฟังก์ชัน `fetch()` กำลังวิ่งทำงานอยู่บนเน็ตเวิร์ก ข้อใดคือแนวปฏิบัติที่ดีในการคุมสเตตเพื่อความพึงพอใจของผู้เรียนหรือผู้ใช้แอป?

ก. ตั้งค่าสเตต `isLoading` เป็น `true` ก่อนเรียกเฟทช์ และสลับเป็น `false` ในบล็อก `finally` พร้อมแสดงคอมโพเนนต์ `ActivityIndicator` ระหว่างรอสายข้อมูล

ข. ปิดปุ่มกดไม่ให้สัมผัส และห้ามแสดงความคืบหน้าใด ๆ บนจอจนกว่าแอปจะค้าง

ค. แสดงกล่องข้อความเตือนภัยสีเหลืองกะพริบบนขอบหน้าต่างจอมือถือ

ง. บังคับย้อนกลับหน้าแรกสุดของแอปพลิเคชันทุกครั้งที่ระบบกำลังรอโหลดข้อมูล

ข้อ 10: ในระบบจัดการจดจำสิทธิ์การล็อกอิน (Auth Token Caching) หากเราต้องการให้โทรศัพท์มือถือทำการเช็คประวัติชื่อล็อกอินในเครื่องเพื่อกระโดดข้ามหน้ากรอกแบบฟอร์ม เราต้องเรียกเขียนโค้ดตรวจสอบโทเค็นในจุดใดของแอป?

ก. เขียนตรวจในฟังก์ชัน `onPress` ของปุ่มล็อกเอาต์

ข. เขียนตรวจในบล็อกสเตต `useEffect` ของหน้าต่างแสดงผลแรกสุด (Root Component) เพื่ออ่านค่าคีย์โทเค็นจาก AsyncStorage ตอนเปิดแอปขึ้นมา

ค. เขียนระบุซ้อนอยู่ในสไตล์ชีทภายนอกเพื่อให้คอมไพเลอร์เช็คสี

ง. เขียนส่งในพารามิเตอร์ `body` ของ POST request แรกสุดตอนสลับสี

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: AsyncStorage เก็บข้อมูลบนเครื่องส่วนตัวปลายทาง (Local Storage) ไม่ใช่ฐานข้อมูลระบบ Cloud

• ข. ถูก: เป็นระบบจัดเก็บบนเครื่องเก็บแบบคีย์-ค่า และทำงานรองรับเฉพาะประเภทข้อมูล String เท่านั้น

• ค. ผิด: เป็นระบบจัดเก็บถาวร (Persistent) ข้อมูลจะไม่สูญหายเมื่อปิดแอปพลิเคชัน

• ง. ผิด: ไม่ได้มีสิทธิ์เข้าถึงหรือบันทึกรูปลงคลังภาพส่วนกลางของแอปกล้องโดยตรงโดยไม่มีสิทธิ์พิเศษ

เฉลย ข้อ 2: ตอบ ข.

• เหตุผลประกอบ:

• **ก. ค. และ ง. ผิด:** ข้อมูลไม่ได้ถูกส่งไปภายนอก และไม่สูญหายเมื่อไฟดับหรือรีสตาร์ทตัวเครื่อง

• **ข. ถูก:** หน้าที่ของ AsyncStorage คือการเขียนไฟล์ข้อมูลลงบนฮาร์ดไดรฟ์ภายในเครื่องแบบถาวร ข้อมูลจึงยังคงปลอดภัยอยู่ครบถ้วน

เฉลย ข้อ 3: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `getItem` ใช้สำหรับอ่านข้อมูล

• **ข. ผิด:** `removeItem` ใช้สำหรับลบข้อมูลชิ้นนั้น ๆ

• **ค. ถูก:** `setItem` ทำงานรับค่าพารามิเตอร์ Key และ Value เพื่อบันทึกเขียนข้อมูลทับหรือบันทึกใหม่

• **ง. ผิด:** `clear` ใช้ล้างลบข้อมูลทุกชิ้นในคลัง

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ข้อมูลออบเจกต์จะไม่ถูกเก็บไว้สมบูรณ์หากไม่ผ่านการแปลงค่าก่อน

• **ข. ถูก:** เนื่องจาก AsyncStorage รับค่าสตริงเท่านั้น หากส่ง JavaScript Object เข้าไปโดยไม่ใช้เครื่องมือแปลง ค่าจะถูกแปลงเป็นรูปข้อความ "`[object Object]`" ส่งผลให้ตอนดึงข้อมูลกลับมา ข้อมูลโครงสร้างคุณสมบัติภายในจะหายไปทั้งหมดใช้งานไม่ได้

• **ค. และ ง. ผิด:** ไม่มีส่วนต่อพ่วงความเชื่อมโยงกับเฟซบุ๊ก หรือการเปิดไฟแอปเตือนภัยทางสี

เฉลย ข้อ 5: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เป็นมาตรฐานสากลในการจัดเก็บออบเจกต์บนระบบ AsyncStorage โดยแปลงเป็นสตริงรูปแบบ JSON ก่อนบันทึก และถอดสตริงข้อความกลับมาเป็น JavaScript Object ใช้งานผ่าน `JSON.parse`

• **ข. ค. และ ง. ผิด:** การทำ `toString` ทั่วไปไม่รองรับโครงสร้างซับซ้อน และ `clear` จะล้างข้อมูลทิ้งทั้งหมด

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Fetch จะสั่ง Reject สัญญาณเฉพาะกรณีเกิดปัญหาการเชื่อมต่อทางกายภาพล้มเหลว (เช่น เน็ตตัด สัญญาณหลุด หรือไม่พบ DNS โดเมน) เท่านั้น

• **ข. ถูก:** รหัสข้อผิดพลาดประเภท HTTP Status (เช่น 404, 500) จากเครื่องเซิร์ฟเวอร์ยังถือว่ารับส่งสำเร็จในทางระบบเชื่อมต่อ เอนจิ้น Fetch จึงยัง Resolve คืนผลลัพธ์มาให้ เราจึงจำเป็นต้องตรวจเช็คค่าความสำเร็จผ่าน `response.ok` ด้วยตนเองเพื่อความปลอดภัย

• **ค. และ ง. ผิด:** ไม่ได้ปิดระบบเน็ตเวิร์ก หรือทำลายข้อมูลโทเค็นในเครื่องโดยอัตโนมัติ

เฉลย ข้อ 7: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** พร็อพส์อื่น ๆ ให้ข้อมูลประเภทข้อมูลหรือส่วนหัว แต่ไม่ได้เก็บสถานะสำเร็จเป็น Boolean

• **ค. ถูก:** `response.ok` เป็นคุณสมบัติประเภท Boolean (true/false) ที่ตัวระบบ Fetch เตรียมให้ตรวจสอบสถานะว่ารหัส HTTP Code ตอบกลับมาจากเซิร์ฟเวอร์เรียบร้อยดีหรือไม่ (มีค่าเป็น true เมื่อรหัสอยู่ในช่วง 200-299)

เฉลย ข้อ 8: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** โครงสร้าง HTTP POST ที่ถูกต้องจำเป็นต้องเปลี่ยน `method` เป็น `'POST'` เติม Headers แจ้งเซิร์ฟเวอร์ว่าเราส่งฟอร์มข้อมูลชนิด JSON และแปลงชุดข้อมูลให้กลายเป็น String ในคีย์ `body`

• **ข. ค. และ ง. ผิด:** ผิดข้อกำหนดด้านวิธี และไม่มีการตั้งค่า Headers ครอบสตรีมสตริงที่ถูกต้อง

เฉลย ข้อ 9: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** การสลับค่าสเตต `isLoading` ตั้งค่าเพื่อควบคุมและเรนเดอร์ไอคอน ActivityIndicator ระหว่างดาวน์โหลดเป็นเทคนิคมาตรฐานในการแจ้งความเคลื่อนไหวเพื่อไม่ให้เครื่องดูค้างนิ่ง

• **ข. ค. และ ง. ผิด:** ไม่ควรปิดความเคลื่อนไหวเงียบโดยไม่มีปฏิกิริยา และไม่ควรกระพริบเตือนตื่นตระหนก หรือบังคับย้ายหน้าจอผู้ใช้ไปมา

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ปุ่มล็อกเอาต์มีจุดประสงค์เพื่อทำลายโทเค็นทิ้ง ไม่ใช่ตรวจสอบสิทธิ์เข้าใช้งาน

• **ข. ถูก:** วงรอบชีวิตการเรนเดอร์เริ่มต้นของคอมโพเนนต์จะกระตุ้นคำสั่ง `useEffect` เป็นด่านแรกสุด การตรวจค้นหาข้อมูล `user_auth_token` ในจุดนี้ จะทำให้เราสลับเปลี่ยนหน้าจอได้ถูกต้องทันทีตั้งแต่แอปเปิดหน้าแรก

• **ค. และ ง. ผิด:** ไฟล์สไตล์ชีทภายนอกควบคุมเฉพาะความสวยงามดีไซน์ และ POST request ส่งข้อมูลหาเซิร์ฟเวอร์ไม่ใช่การดึงตรวจค่าในโทรศัพท์

ปฏิบัติการท้ายบทเรียน: การสร้างแอปบันทึกข้อคิดเห็นออนไลน์แบบจดจำประวัติ

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้งานหน่วยความจำถาวรภายในเครื่องร่วมกับการจัดการส่งต่อข้อมูลขึ้นสู่คลาวด์ผ่านเครือข่าย โดยผู้เรียนจะได้สร้างแอปพลิเคชัน **"สมุดโน้ตบันทึกและซิงก์ความคิดเห็นออนไลน์ (Synchronized Note Keeper)"** ซึ่งจะทำหน้าจอดึงข้อมูลบันทึกส่วนกลางมาจาก REST API ด้วย HTTP GET แสดงผลในลิสต์ และเปิดฟอร์มเขียนเพิ่มโพสต์ส่งกลับขึ้นเซิร์ฟเวอร์ด้วย HTTP POST พร้อมเพิ่มความเสถียรโดยการบันทึกสำรอง (Cache) ข้อมูลรอบล่าสุดเก็บลงเครื่องผ่าน **`AsyncStorage`** เพื่อให้อ่านประวัติโน้ตเก่าขึ้นมารายงานหน้าต่างจอได้ทันที แม้กรณีสัญญาณอินเทอร์เน็ตขาดหาย

1. วัตถุประสงค์การเรียนรู้

1. สามารถติดตั้งและเขียนโปรแกรมดึงข้อมูลแบบออฟไลน์แคชชิ่ง (Offline-First caching) โดยเรียกอ่านประวัติเก่าจาก AsyncStorage ขึ้นมาจัดแสดงเมื่อแอปเริ่มต้นทำงาน

2. สามารถเขียนระบบดักส่งข้อมูลด้วย Fetch API (HTTP POST Method) เพื่อบันทึกโน้ตใหม่ขึ้นสู่เซิร์ฟเวอร์ภายนอกสำเร็จ

3. สามารถจัดสร้างสถานะ Loading และจัดการป๊อปอัพ Alert แจ้งเตือนข้อผิดพลาดกรณีการติดต่อเครือข่ายล้มเหลวได้อย่างถูกต้อง

4. สามารถสลักแปลงข้อมูลอาร์เรย์ของโพสต์แบบซับซ้อนให้กลายเป็น JSON String และแกะกลับคืนเพื่อจัดสรรร่วมกับคอมโพเนนต์ **`FlatList`** ได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องเขียนสร้างแอปพลิเคชันที่มีโครงสร้างองค์ประกอบและการทำงานตามเงื่อนไขดังนี้:

1. การทำงานร่วมกับเซิร์ฟเวอร์ภายนอก (Fetch API Integration):

• **ดึงข้อมูลเริ่มต้น (GET Request):** เมื่อเปิดแอปขึ้นมาเป็นครั้งแรก ให้ทำการยิง GET request ไปที่:

```
GET https://jsonplaceholder.typicode.com/posts?_limit=5
```

เพื่อดึงรายการโน้ตจำนวน 5 โพสต์ล่าสุดมาแสดงผล

• **ออฟไลน์แคชชิ่ง (Local Caching):** เมื่อดึงข้อมูลสำเร็จ ให้บันทึกชุดข้อมูลอาร์เรย์นี้เก็บสำรองถาวรลงใน `AsyncStorage` ภายใต้คีย์ `'cached_notes'` เสมอ

• **การส่งข้อมูลโพสต์ใหม่ (POST Request):** มีแบบฟอร์มให้กรอก "หัวข้อโน้ต (Title)" และ "รายละเอียดเนื้อหา (Content)" เมื่อกดปุ่มส่งข้อมูลให้ยิง POST request ไปที่:

```
POST https://jsonplaceholder.typicode.com/posts
```

โดยเมื่อได้ผลลัพธ์ตอบกลับสำเร็จ (Response Code: 201) ให้นำรายการที่บันทึกสำเร็จสมมุตินั้น ไปพ่วงแทรกต่อเพิ่มไว้ด้านบนสุดของรายการแสดงผลในหน้าจอทันที เพื่อสะท้อนภาพข้อมูลอัปเดตแบบเรียลไทม์

2. ระบบต้านทานสัญญาณอินเทอร์เน็ตล่ม (Offline/Error Resilience):

• หากสมาร์ทโฟนปิดสัญญาณเน็ตหรือเซิร์ฟเวอร์พังจน Fetch ล้มเหลว แอปพลิเคชันต้องไม่ดับพัง แต่ต้องมีกลไกดึงข้อมูลโน้ตที่แคชไว้ล่าสุดใน AsyncStorage ขึ้นมาทำงานแทนที่บนลิสต์ และแจ้งเตือนให้ผู้ใช้งานทราบผ่านคำสั่ง `Alert.alert` ว่า "กำลังทำงานในโหมดออฟไลน์"

3. การแสดงผล UI สไตล์โมเดิร์น:

• จัดสไตล์หน้าจอเป็นโทนสี Slate/Sky หรูหราสะอาดตา มีช่องรับข้อมูล Text inputs ของหัวเรื่องและเนื้อหาแยกกัน มีการแสดงผลตัวหมุน ActivityIndicator ในระหว่างเชื่อมเน็ตเวิร์กและการ์ดแสดงผลโน้ตย่อย

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งแพ็คเกจเสริมสำหรับเก็บข้อมูล

สั่งรันคำสั่งติดตั้งคลังหน่วยเก็บข้อมูลถาวรภายในเครื่อง:

```
npx expo install @react-native-async-storage/async-storage
```

ขั้นตอนที่ 2: สร้างฟังก์ชันสำหรับอ่านและเขียนแคช

เขียนแยกโค้ดจัดการข้อมูล JSON เพื่อให้อ่านเข้าใจง่าย:

```
import AsyncStorage from '@react-native-async-storage/async-storage';

const CACHE_KEY = 'cached_notes';

// บันทึกอาร์เรย์ของโพสต์ลงเครื่อง
const cacheNotes = async (notes: any[]) => {
  try {
    await AsyncStorage.setItem(CACHE_KEY, JSON.stringify(notes));
  } catch (e) {
```

```
    console.error('ไม่สามารถแคชข้อมูลได้:', e);
  }
};

// กู้ประวัติอาร์เรย์จากเครื่อง
const getCachedNotes = async () => {
  try {
    const raw = await AsyncStorage.getItem(CACHE_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch (e) {
    console.error('ไม่สามารถดึงข้อมูลแคชได้:', e);
    return [];
  }
};
```

ขั้นตอนที่ 3: จัดการกระแสไหลของสเตต (State Workflow)

ประกาศสเตตควบคุม **notes** (เก็บข้อมูลโพสต์), **isLoading** (เช็คเวลาโหลด), และ **isSubmitting** (เช็คเวลาส่งข้อมูล POST) และจัดทำ Logic คัดกรองเครือข่าย:

```
const loadInitialData = async () => {
  try {
    setIsLoading(true);
    const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=5');

    if (response.ok) {
      const data = await response.json();
      setNotes(data);
    await cacheNotes(data); // บันทึกสำรองลง Storage
    } else {
      throw new Error('การตอบรับจากเซิร์ฟเวอร์ไม่สมบูรณ์');
    }
  } catch (error) {
  // กรณีเน็ตหลุด ให้กู้ข้อมูลเก่าขึ้นมาโชว์ตัว
    const localData = await getCachedNotes();
    setNotes(localData);
    Alert.alert('โหมดออฟไลน์', 'ไม่สามารถเชื่อมต่อเซิร์ฟเวอร์ได้ ดึงข้อมูลล่าสุดที่บันทึกไว้ในเครื่องขึ้นมาแสดงผลแทน');
  } finally {
    setIsLoading(false);
  }
};
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกซอร์สโค้ดของแอปพลิเคชันแบบหน้าต่างเดียวชุดนี้ ไปจัดทำและบันทึกทับในไฟล์หลักของโครงการ (เช่น `app/index.tsx` หรือ `App.tsx`) เพื่อรันและจำลองการดึงส่งข้อมูลเครือข่าย:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  FlatList,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  KeyboardAvoidingView,
  Platform
} from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

// โมเดลข้อมูลโน้ตข้อความ
interface Note {
  id: number;
  title: string;
  body: string;
}

const STORAGE_KEY = 'cached_notes';

export default function SynchronizedNoteKeeper() {
  const [notes, setNotes] = useState<Note[]>([]);
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

 // 1. ตรวจจับข้อมูลตอนเปิดหน้าจอแรกสุด
  useEffect(() => {
    fetchNotes();
  }, []);

 // 2. ฟังก์ชันหลักในการดึงรายการโน้ต (GET)
  const fetchNotes = async () => {
    try {
      setIsLoading(true);
    // ส่งคำขอดึงข้อมูลจำกัดจำนวน 5 โพสต์ล่าสุด
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=5');
```

```
      if (response.ok) {
        const remoteData = await response.json();
        setNotes(remoteData);
    // ทำการเซฟสำรองเก็บในเครื่องเป็น Offline Cache
      await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(remoteData));
      } else {
        throw new Error('เซิร์ฟเวอร์ขัดข้อง');
      }
    } catch (error) {
   // ดักจับกรณีออฟไลน์หรือเน็ตพัง ให้ดึงจาก AsyncStorage
      const localData = await AsyncStorage.getItem(STORAGE_KEY);
      if (localData) {
        setNotes(JSON.parse(localData));
      }
      Alert.alert(
        'สัญญาณขาดหาย',
        'ไม่สามารถดึงข้อมูลจากอินเทอร์เน็ตได้ ระบบทำการดึงข้อมูลล่าสุดที่บันทึกในเครื่องขึ้นแสดงผลทดแทน'
   );
    } finally {
      setIsLoading(false);
    }
  };

// 3. ฟังก์ชันการส่งข้อมูลโน้ตใหม่ (POST)
  const handleAddNote = async () => {
    if (!title.trim() || !content.trim()) {
      Alert.alert('กรอกข้อมูลไม่ครบ', 'กรุณากรอกหัวข้อโน้ตและรายละเอียดให้ครบถ้วน');
      return;
    }

    try {
      setIsSubmitting(true);

   // สั่งส่งข้อมูลชนิด POST ไปเก็บที่ API ปลายทาง
      const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
    headers: {
          'Content-Type': 'application/json; charset=UTF-8',
        },
        body: JSON.stringify({
          title: title,
          body: content,
          userId: 1
        }),
      });

      if (response.ok) {
        const newNoteFromServer = await response.json();
```

```
    // จำลองโครงสร้างการซิงก์โดยนำโน้ตใหม่ไปต่อไว้ด้านบนสุดของสเตตลิสต์ปัจจุบัน
        const updatedNotes = [
          {
            id: newNoteFromServer.id, // รหัสไอดี 101 ที่ได้คืนมาจาก REST API จำลอง
           title: title,
            body: content
          },
     ...notes
    ];

        setNotes(updatedNotes);

    // อัปเดตไฟล์แคชออฟไลน์ในเครื่องให้มีโน้ตใหม่ติดตัวไปด้วย
        await AsyncStorage.setItem(STORAGE_KEY,
JSON.stringify(updatedNotes));

        Alert.alert('สำเร็จ!', 'บันทึกโน้ตซิงก์ขึ้นระบบเซิร์ฟเวอร์เรียบร้อย');
        setTitle('');
        setContent('');
      } else {
        throw new Error('เกิดข้อผิดพลาดในการประมวลผลบนคลาวด์');
      }
    } catch (error) {
      Alert.alert('ส่งไม่สำเร็จ', 'ไม่สามารถบันทึกข้อมูลขึ้นเซิร์ฟเวอร์ได้ กรุณาเชื่อมต่ออินเทอร์เน็ต');
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <KeyboardAvoidingView
        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
        style={styles.container}
      >
        {/* หัวข้อแอป */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>□ Synchronized Note
Keeper</Text>
          <Text style={styles.headerSubtitle}>ซิงก์บันทึกผ่าน API และแคชข้อมูลออฟไลน์
ถาวร</Text>
        </View>

        {/* แบบฟอร์มเขียนบันทึกส่งข้อมูล */}
        <View style={styles.formCard}>
          <Text style={styles.formLabel}>เขียนโน้ตชิ้นใหม่</Text>
          <TextInput
            style={styles.input}
            placeholder="หัวข้อโน้ต..."
```

```
            placeholderTextColor="#64748b"
            value={title}
            onChangeText={setTitle}
      />
          <TextInput
        style={[styles.input, styles.textArea]}
            placeholder="รายละเอียดเนื้อหา..."
            placeholderTextColor="#64748b"
            value={content}
            onChangeText={setContent}
            multiline
      />

      {isSubmitting ? (
            <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
      ) : (
            <TouchableOpacity style={styles.submitButton}
onPress={handleAddNote}>
              <Text style={styles.submitButtonText}>ชิงก์โพสต์ขึ้นคลาวด์</Text>
            </TouchableOpacity>
      )}
        </View>

        {/* ส่วนแสดงรายการข้อมูลโน้ต */}
        <View style={styles.listContainer}>
          <View style={styles.listHeaderRow}>
            <Text style={styles.listTitle}>บันทึกของคุณในเครื่อง</Text>
            <TouchableOpacity onPress={fetchNotes} disabled={isLoading}>
              <Text style={styles.refreshText}>{isLoading ? 'กำลังโหลด...' :
'🔄 รีเฟรชล่าสุด'}</Text>
            </TouchableOpacity>
      </View>

          {isLoading ? (
            <View style={styles.centerLoading}>
              <ActivityIndicator size="large" color="#38bdf8" />
              <Text style={styles.loadingText}>กำลังติดต่อดึงข้อมูล API...</Text>
            </View>
      ) : (
            <FlatList
              data={notes}
              keyExtractor={(item, index) => item.id.toString() + index}
              renderItem={({ item }) => (
                <View style={styles.noteCard}>
                  <Text style={styles.noteCardTitle}>📌
{item.title}</Text>
                  <Text style={styles.noteCardBody}>{item.body}</Text>
                  <View style={styles.badgeRow}>
```

```
                    <View style={styles.syncBadge}>
                      <Text style={styles.badgeText}>ID: {item.id}
(Synced)</Text>
                    </View>
                  </View>
                </View>
        )}
              ListEmptyComponent={
                <View style={styles.emptyContainer}>
                  <Text style={styles.emptyText}>ไม่มีข้อมูลบันทึกในเครื่องขณะนี้</Text>
                </View>
              }
      />
     )}
        </View>
       </KeyboardAvoidingView>
    </SafeAreaView>
 );
}

const styles = StyleSheet.create({
  safeArea: {
   flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
    padding: 16,
  },
  header: {
    alignItems: 'center',
    marginVertical: 12,
  },
  headerTitle: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
```

```
    marginBottom: 16,
  },
  formLabel: {
   color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 14,
    marginBottom: 10,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
    color: '#ffffff',
    padding: 12,
    fontSize: 14,
    marginBottom: 10,
  },
  textArea: {
    height: 70,
    textAlignVertical: 'top',
  },
  submitButton: {
    backgroundColor: '#38bdf8',
    borderRadius: 8,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
  },
  submitButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
    paddingVertical: 10,
  },
  listContainer: {
    flex: 1,
  },
  listHeaderRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 10,
  },
  listTitle: {
    fontSize: 16,
```

```
    fontWeight: 'bold',
    color: '#ffffff',
  },
  refreshText: {
    color: '#38bdf8',
    fontSize: 13,
  },
  centerLoading: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 10,
    fontSize: 13,
  },
  noteCard: {
    backgroundColor: '#1e293b',
    borderRadius: 12,
    padding: 14,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 12,
  },
  noteCardTitle: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 15,
    textTransform: 'capitalize',
    marginBottom: 6,
  },
  noteCardBody: {
    color: '#cbd5e1',
    fontSize: 13,
    lineHeight: 18,
  },
  badgeRow: {
    flexDirection: 'row',
    marginTop: 10,
  },
  syncBadge: {
    backgroundColor: '#0284c7',
    paddingVertical: 2,
    paddingHorizontal: 8,
    borderRadius: 4,
  },
  badgeText: {
```

```
    color: '#ffffff',
    fontSize: 10,
    fontWeight: 'bold',
  },
  emptyContainer: {
    alignItems: 'center',
    paddingVertical: 40,
  },
  emptyText: {
    color: '#64748b',
    fontSize: 14,
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Using SQLite*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/

MDN Web Docs. (2024). *Using the Fetch API*. จาก https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

React Native Async Storage. (2024). *Async Storage Documentation*. จาก https://react-native-async-storage.github.io/async-storage/docs/install/